TRƯỜNG ĐẠI HỌC PHÚ XUÂN
KHOA CÔNG NGHỆ THÔNG TIN

HƯỚNG DẪN THỰC HÀNH
INT.7.18 — WEB FRONTEND NÂNG CAO

BÀI 8

BIỂU MẪU CÓ KIỂM SOÁT, XỬ LÝ DỮ LIỆU NHẬP VÀ ĐỒNG BỘ TRẠNG THÁI

Hạng mục	Nội dung
Dự án	phu-xuan-react — nhánh feature/buoi8-form-co-kiem-soat
Công nghệ + phiên bản	Node.js 22 LTS · Vite 6 · React 19.1 · (không dùng thư viện quản lý biểu mẫu ngoài — tự viết bằng React thuần)
Mã học phần — Trường	INT.7.18 — Trường Đại học Phú Xuân, Khoa Công nghệ Thông tin
Thời lượng trên lớp	60 phút (Lab 1, 2, 3) — 20 giờ dạy học Bài 8/20, dạng A
Bài tập về nhà	Lab 4, Lab 5 — nộp cùng thời hạn với bài tập về nhà mục XVIII của giáo án lý thuyết

Đi kèm tài liệu Lý thuyết Bài 8. Đọc lại mục VI–XV của tài liệu lý thuyết nếu quên cú pháp trong lúc làm lab.

MỤC TIÊU THỰC HÀNH BÀI 8

Sau Bài thực hành, sinh viên sẽ:

1. Dựng được biểu mẫu "Thêm địa điểm tham quan" với đủ sáu ô nhập có kiểm soát, dùng một handler duy nhất cho toàn bộ.
2. Xử lý đúng từng loại ô nhập: văn bản, vùng văn bản, danh sách chọn, hộp kiểm đơn, nhóm nút chọn.
3. Viết hàm kiểm chứng dữ liệu thuần và gắn thông báo lỗi hiển thị theo trạng thái đã chạm (touched).
4. Quản lý trạng thái gửi biểu mẫu bằng một biến nhiều giá trị, khoá nút đúng lúc, không mất dữ liệu khi gửi thất bại.
5. ⭐ Rút toàn bộ logic biểu mẫu vào hook tùy biến useForm và chứng minh hook dùng lại được cho một biểu mẫu thứ hai.
6. ⭐ Đồng bộ hai thành phần anh em bằng kỹ thuật nâng trạng thái lên (lifting state up).

Ánh xạ Lab với CLO

Lab	Trọng tâm	CLO
Lab 1 (trên lớp)	Biểu mẫu có kiểm soát cơ bản — ô văn bản, vùng văn bản, danh sách chọn	CLO1
Lab 2 (trên lớp)	Hộp kiểm, nhóm nút chọn, nhóm hộp kiểm nhiều lựa chọn	CLO1, CLO3
Lab 3 (trên lớp)	Kiểm chứng dữ liệu, trạng thái đã chạm, trạng thái gửi	CLO1, CLO3, CLO5
Lab 4 (về nhà)	Hook useForm dùng lại cho biểu mẫu thứ hai	CLO3, CLO5
Lab 5 (về nhà)	Nâng trạng thái lên cho khung xem trước trực tiếp	CLO1, CLO3

YÊU CẦU TRƯỚC KHI BẮT ĐẦU

Kiểm tra trước khi mở máy

[ ] Đã kéo về nhánh main mới nhất và tạo nhánh feature/buoi8-form-co-kiem-soat từ trạng thái cuối Bài 7.

[ ] Chạy được npm run dev và thấy trang chủ dự án phu-xuan-react hiển thị bình thường tại localhost:5173.

[ ] Bộ quản lý sự kiện và preventDefault của Bài 7 vẫn còn chạy được — mở lại component đã làm Bài trước để kiểm tra nhanh.

[ ] Đã cài React DevTools và có thể mở tab Components trên trình duyệt.

[ ] Đã tạo thư mục src/features/dia-diem/ và src/hooks/ (tạo trống nếu chưa có).

LAB 1 — DỰNG BIỂU MẪU CÓ KIỂM SOÁT CƠ BẢN

⏱ **Thời gian: 15 phút | 🎯 Mục tiêu: dựng khung biểu mẫu "Thêm địa điểm tham quan" với ba ô đầu tiên (tên, mô tả, phường) ở dạng có kiểm soát, dùng một state đối tượng và một handler duy nhất.**

Bước 1 — Tạo tệp thành phần và khai báo state

Tạo tệp src/features/dia-diem/FormThemDiaDiem.jsx. Khai báo hằng số giá trị ban đầu bên ngoài component (để không bị tạo lại object mới mỗi lần kết xuất), rồi khai báo state.

JSX — src/features/dia-diem/FormThemDiaDiem.jsx

```jsx
import { useState } from 'react';

const GIA_TRI_BAN_DAU = {
  ten: '',
  moTa: '',
  giaVe: '',
  phuong: '',
  loaiHinh: 'di-tich',
  dongY: false,
};

export default function FormThemDiaDiem() {
  const [duLieu, setDuLieu] = useState(GIA_TRI_BAN_DAU);

  return (
    <form>
      {/* các ô nhập sẽ thêm ở Bước 2 */}
    </form>
  );
}
```

Kết quả mong đợi

Trang trắng, chỉ có một khung <form> rỗng — chưa có lỗi Console.
Mở React DevTools > Components > FormThemDiaDiem, thấy state duLieu = { ten: "", moTa: "", ... }

Bước 2 — Viết một handler duy nhất cho mọi ô nhập

Đây là hàm quan trọng nhất của cả biểu mẫu. Gõ đúng nguyên văn — khoá tính toán `[name]` và cú pháp trải rộng `...truoc` bắt buộc phải có.

JSX — thêm vào trong component, trước câu lệnh return

```
function xuLyThayDoi(e) {
  const { name, value, type, checked } = e.target;

  setDuLieu((truoc) => ({
    ...truoc,
    [name]: type === 'checkbox' ? checked : value,
  }));
}
```

Bước 3 — Gắn ba ô nhập đầu tiên

JSX — thay phần return ở Bước 1 bằng đoạn dưới đây

```
return (
  <form>
    <div className="truong">
      <label htmlFor="ten">Tên địa điểm</label>
      <input
        id="ten"
        name="ten"
        type="text"
        value={duLieu.ten}
        onChange={xuLyThayDoi}
        placeholder="Ví dụ: Lăng Minh Mạng"
      />
    </div>

    <div className="truong">
      <label htmlFor="moTa">Mô tả ngắn</label>
      <textarea
        id="moTa"
        name="moTa"
        rows={4}
        value={duLieu.moTa}
        onChange={xuLyThayDoi}
      />
    </div>

    <div className="truong">
      <label htmlFor="phuong">Phường / xã</label>
      <select id="phuong" name="phuong" value={duLieu.phuong} onChange={xuLyThayDoi}>
        <option value="">-- Chọn phường --</option>
        <option value="phu-hau">Phú Hậu</option>
        <option value="huong-long">Hương Long</option>
        <option value="thuy-bieu">Thuỷ Biều</option>
        <option value="vy-da">Vỹ Dạ</option>
      </select>
    </div>

    {/* Dòng debug tạm thời — xoá ở Bước 4 */}
    <pre>{JSON.stringify(duLieu, null, 2)}</pre>
  </form>
);
```

Kết quả mong đợi

Gõ "Lăng Minh Mạng" vào ô Tên -> khối <pre> bên dưới cập nhật ngay theo từng ký tự:

```
{
  "ten": "Lăng Minh Mạng",
```

```
  "moTa": "",
  "giaVe": "",
  "phuong": "",
  "loaiHinh": "di-tich",
  "dongY": false
}
```

Bước 4 — Xác nhận và dọn debug

- Gõ thử vào cả ba ô, quan sát khối debug cập nhật đúng theo từng ký tự.
- Chọn một phường trong danh sách, xác nhận state.phuong đổi đúng giá trị (không phải nhãn hiển thị).
- Xoá khối `<pre>{JSON.stringify(...)}</pre>` — chỉ dùng để debug, không phải sản phẩm cuối cùng.

> **✓ CHECKPOINT: Lab 1 hoàn thành**
>
> [ ] Ba ô nhập (tên, mô tả, phường) đều thay đổi đúng theo state khi gõ hoặc chọn.
>
> [ ] Không còn cảnh báo màu đỏ nào trong Console liên quan tới value hay onChange.
>
> [ ] React DevTools cho thấy đúng một biến state duy nhất (duLieu) chứa toàn bộ dữ liệu biểu mẫu.

Vấn đề gặp phải trong Lab 1

Vấn đề gặp phải	Nguyên nhân có thể	Cách khắc phục
Gõ vào ô Tên nhưng không thấy chữ xuất hiện	Thiếu `onChange={xuLyThayDoi}` trên ô input	Kiểm tra lại đủ ba thuộc tính: id, name, value, onChange trên mỗi ô
Chọn phường nhưng ô select tự nhảy về "-- Chọn phường --"	Thuộc tính `name="phuong"` bị gõ sai chính tả, không khớp khoá trong GIA_TRI_BAN_DAU	Đối chiếu chính xác tên khoá giữa state và thuộc tính name của từng ô
Console báo "a component is changing an uncontrolled input"	Một khoá trong GIA_TRI_BAN_DAU bị bỏ sót hoặc gõ nhầm tên	Kiểm tra đủ 6 khoá đúng chính tả trong GIA_TRI_BAN_DAU

LAB 2 — HỘP KIỂM, NÚT CHỌN VÀ NHÓM HỘP KIỂM

⏱ **Thời gian: 15 phút | 🎯 Mục tiêu: thêm ba ô nhập còn lại (giá vé, loại hình, xác nhận) và một nhóm hộp kiểm tiện ích, hoàn thiện toàn bộ sáu+ ô của biểu mẫu.**

Bước 1 — Ô giá vé (input number giữ dạng chuỗi)

JSX — thêm vào form, sau ô "Mô tả ngắn"

```
<div className="truong">
 <label htmlFor="giaVe">Giá vé (VNĐ)</label>
 <input
  id="giaVe"
  name="giaVe"
  type="number"
  value={duLieu.giaVe}
  onChange={xuLyThayDoi}
  placeholder="0"
 />
</div>
```

Lưu ý: `e.target.value` của ô số vẫn luôn là chuỗi. Không ép `Number()` ở bước này — giữ nguyên chuỗi trong state, việc ép số để dành cho lúc gửi (Lab 3).

Bước 2 — Nhóm nút chọn "Loại hình"

JSX — thêm vào form, sau ô "Phường / xã"

```
<fieldset>
 <legend>Loại hình</legend>

 <label>
  <input
   name="loaiHinh"
   type="radio"
   value="di-tich"
   checked={duLieu.loaiHinh === 'di-tich'}
   onChange={xuLyThayDoi}
  />
  Di tích lịch sử
 </label>

 <label>
  <input
   name="loaiHinh"
   type="radio"
   value="am-thuc"
   checked={duLieu.loaiHinh === 'am-thuc'}
   onChange={xuLyThayDoi}
  />
  Điểm ẩm thực
 </label>
</fieldset>
```

> **Vì sao dùng cùng handler xuLyThayDoi cho cả radio?**
> Nút chọn không có thuộc tính `checkbox` trong `type === "checkbox"` của handler, nên nó rơi vào nhánh else và lưu thẳng `value` — đúng như mong muốn, không cần viết handler riêng.

Bước 3 — Hộp kiểm đơn "Xác nhận"

JSX — thêm vào cuối form, trước nút Gửi

```
<label>
 <input
  name="dongY"
  type="checkbox"
  checked={duLieu.dongY}
  onChange={xuLyThayDoi}
 />
 Tôi xác nhận thông tin địa điểm là chính xác
</label>
```

Bước 4 — Nhóm hộp kiểm nhiều lựa chọn: Tiện ích

Tiện ích là danh sách, nên cần một state mảng riêng, tách khỏi state đối tượng duLieu để không làm phức tạp handler chính.

JSX — thêm state mới ngay dưới useState(GIA_TRI_BAN_DAU) và một handler riêng

```
const [tienIch, setTienIch] = useState([]);

const DS_TIEN_ICH = [
 { ma: 'bai-xe', ten: 'Bãi đỗ xe' },
 { ma: 'huong-dan', ten: 'Có hướng dẫn viên' },
 { ma: 've-online', ten: 'Bán vé trực tuyến' },
 { ma: 'khu-ve-sinh', ten: 'Khu vệ sinh công cộng' },
];

function xuLyTich(e) {
 const { value, checked } = e.target;
 setTienIch((truoc) =>
  checked ? [...truoc, value] : truoc.filter((m) => m !== value)
 );
}
```

JSX — thêm vào form, sau nhóm "Loại hình"

```
<fieldset>
 <legend>Tiện ích tại điểm đến</legend>
 {DS_TIEN_ICH.map((ti) => (
  <label key={ti.ma}>
   <input
    type="checkbox"
    value={ti.ma}
    checked={tienIch.includes(ti.ma)}
    onChange={xuLyTich}
   />
   {ti.ten}
  </label>
 ))}
</fieldset>
```

Kết quả mong đợi

```
Tích "Bãi đỗ xe"            -> tienIch = ["bai-xe"]
Tích thêm "Bán vé trực tuyến" -> tienIch = ["bai-xe", "ve-online"]
Bỏ tích "Bãi đỗ xe"         -> tienIch = ["ve-online"]
```

> **✓ CHECKPOINT: Lab 2 hoàn thành**
>
> [ ] Ô giá vé nhận số, xoá được về rỗng mà không nhảy số lạ.

> [ ] Chỉ một trong hai nút "Di tích lịch sử" / "Điểm ẩm thực" được chọn tại một thời điểm.
> [ ] Hộp kiểm "Xác nhận" tích được và bỏ tích được bình thường.
> [ ] Tích nhiều tiện ích cùng lúc, mảng tienIch cập nhật đúng thứ tự thêm/bớt (kiểm tra trong DevTools).

Vấn đề gặp phải trong Lab 2

Vấn đề gặp phải	Nguyên nhân có thể	Cách khắc phục
Chọn được cả hai nút "Di tích" và "Ẩm thực" cùng lúc	Hai nút không cùng `name`, trình duyệt coi là hai nhóm độc lập	Đặt cùng `name="loaiHinh"` cho mọi nút trong nhóm
Tích hộp kiểm "Xác nhận" nhưng không thấy dấu tích, rồi Console báo lỗi	Dùng `value` thay vì `checked` cho hộp kiểm	Hộp kiểm và nút chọn luôn dùng `checked`, không dùng `value` để điều khiển
Tích một tiện ích thì toàn bộ tiện ích khác bị tích theo	Dùng chung một biến `checked` thay vì `tienIch.includes(ti.ma)` cho từng ô	Mỗi hộp kiểm trong nhóm phải tự so sánh mã của chính nó với mảng state

LAB 3 — KIỂM CHỨNG DỮ LIỆU VÀ TRẠNG THÁI GỬI

⏱ **Thời gian: 15 phút | 🎯 Mục tiêu: viết hàm kiểm chứng dữ liệu, hiển thị lỗi theo trạng thái đã chạm, và quản lý trạng thái gửi biểu mẫu bằng một biến bốn giá trị.**

Bước 1 — Tách hàm kiểm chứng ra tệp riêng

Tạo tệp `src/features/dia-diem/kiemChung.js`. Đây là một hàm thuần: không gọi setState, chỉ nhận dữ liệu và trả về đối tượng lỗi.

JS — src/features/dia-diem/kiemChung.js

```
export function kiemChung(duLieu) {
  const loi = {};

  const ten = duLieu.ten.trim();
  if (!ten) {
    loi.ten = 'Vui lòng nhập tên địa điểm.';
  } else if (ten.length < 3) {
    loi.ten = 'Tên địa điểm phải có ít nhất 3 ký tự.';
  }

  if (duLieu.giaVe === '') {
    loi.giaVe = 'Vui lòng nhập giá vé (nhập 0 nếu miễn phí).';
  } else if (Number.isNaN(Number(duLieu.giaVe)) || Number(duLieu.giaVe) < 0) {
    loi.giaVe = 'Giá vé phải là một số không âm.';
  }

  if (!duLieu.phuong) {
    loi.phuong = 'Vui lòng chọn phường hoặc xã.';
  }

  if (!duLieu.dongY) {
    loi.dongY = 'Bạn cần xác nhận thông tin là chính xác trước khi gửi.';
  }

  return loi;   // rỗng nghĩa là dữ liệu hợp lệ
}
```

Kết quả mong đợi

```
kiemChung({ ten: "", giaVe: "", phuong: "", dongY: false, ... })
-> { ten: "Vui lòng nhập tên địa điểm.", giaVe: "Vui lòng nhập giá vé...", phuong: "...", dongY: "..." }

kiemChung({ ten: "Chùa Thiên Mụ", giaVe: "0", phuong: "huong-long", dongY: true, ... })
-> {}   (đối tượng rỗng — hợp lệ)
```

Bước 2 — Thêm state daCham và tính lỗi khi kết xuất

JSX — trong FormThemDiaDiem.jsx, thêm dưới import và trước hàm xuLyThayDoi

```
import { kiemChung } from './kiemChung';
// ... (các import và state cũ giữ nguyên)

  const [daCham, setDaCham] = useState({});

  // Lỗi là trạng thái dẫn xuất — tính lại mỗi lần kết xuất, KHÔNG lưu thành state
  const loi = kiemChung(duLieu);
```

```
function xuLyRoiO(e) {
  const { name } = e.target;
  setDaCham((truoc) => ({ ...truoc, [name]: true }));
}

function loiHienThi(ten) {
  return daCham[ten] ? loi[ten] : undefined;
}
```

Bước 3 — Gắn onBlur và thông báo lỗi cho ô "Tên"

JSX — sửa lại ô Tên đã viết ở Lab 1

```
<div className="truong">
  <label htmlFor="ten">Tên địa điểm</label>
  <input
    id="ten"
    name="ten"
    type="text"
    value={duLieu.ten}
    onChange={xuLyThayDoi}
    onBlur={xuLyRoiO}
    aria-invalid={loiHienThi('ten') ? true : undefined}
  />
  {loiHienThi('ten') && (
    <p role="alert" className="thong-bao-loi">{loiHienThi('ten')}</p>
  )}
</div>
```

Lặp lại đúng khuôn này cho ba ô còn lại đang có luật kiểm chứng: giá vé, phường, và hộp kiểm xác nhận (hộp kiểm hiển thị lỗi ngay dưới nhãn, không cần onBlur vì người dùng tương tác bằng cách bấm chứ không gõ).

Kết quả mong đợi

Nhấp vào ô Tên rồi rời đi ngay (chưa gõ gì) -> hiện "Vui lòng nhập tên địa điểm."
Gõ "Lă" rồi rời ô -> hiện "Tên địa điểm phải có ít nhất 3 ký tự."
Gõ đủ "Lăng Minh Mạng" -> thông báo lỗi biến mất

Bước 4 — Trạng thái gửi bốn giá trị và hàm xử lý gửi

JSX — thêm state trangThai và hàm xuLyGui

```
const [trangThai, setTrangThai] = useState('cho'); // cho | dang-gui | thanh-cong | that-bai

async function xuLyGui(e) {
  e.preventDefault();

  // Đánh dấu mọi ô là đã chạm để lộ hết lỗi còn sót
  const tatCaDaCham = {};
  Object.keys(GIA_TRI_BAN_DAU).forEach((k) => { tatCaDaCham[k] = true; });
  setDaCham(tatCaDaCham);

  if (Object.keys(kiemChung(duLieu)).length > 0) return;

  try {
    setTrangThai('dang-gui');
    await new Promise((giai) => setTimeout(giai, 1200)); // giả lập gọi máy chủ
    setTrangThai('thanh-cong');
    setDuLieu(GIA_TRI_BAN_DAU);
    setDaCham({});
  } catch {
```

```
    setTrangThai('that-bai');
  }
}
```

Bước 5 — Gắn onSubmit và nút gửi có trạng thái

JSX — sửa thẻ form và thêm nút gửi ở cuối

```
<form onSubmit={xuLyGui} noValidate>
  {/* ... toàn bộ các ô nhập đã viết ở Lab 1 và Lab 2 ... */}

  {trangThai === 'thanh-cong' && (
    <p className="thong-bao-thanh-cong" role="status">Đã thêm địa điểm thành công!</p>
  )}
  {trangThai === 'that-bai' && (
    <p className="thong-bao-loi" role="alert">Có lỗi khi gửi, vui lòng thử lại.</p>
  )}

  <button type="submit" disabled={trangThai === 'dang-gui'}>
    {trangThai === 'dang-gui' ? 'Đang lưu...' : 'Thêm địa điểm'}
  </button>
  <button type="button" onClick={() => { setDuLieu(GIA_TRI_BAN_DAU); setDaCham({}); }}>
    Nhập lại
  </button>
</form>
```

Kết quả mong đợi

Bấm "Thêm địa điểm" khi biểu mẫu còn trống
-> mọi ô hiện thông báo lỗi cùng lúc, KHÔNG gọi hàm giả lập máy chủ

Điền đủ và hợp lệ, bấm "Thêm địa điểm"
-> nút đổi chữ "Đang lưu..." và khoá trong 1.2 giây
-> sau đó hiện "Đã thêm địa điểm thành công!" và biểu mẫu đặt lại về rỗng

✓ CHECKPOINT: Lab 3 hoàn thành

[ ] Bấm gửi khi biểu mẫu còn thiếu dữ liệu: mọi lỗi hiện ra cùng lúc, không có yêu cầu giả lập nào được gọi.

[ ] Rời một ô còn trống (không gõ gì) rồi quay lại gõ đúng: thông báo lỗi của riêng ô đó biến mất, không ảnh hưởng ô khác.

[ ] Trong lúc "dang-gui", nút Thêm địa điểm bị khoá và đổi chữ; bấm thêm lần nữa không tạo yêu cầu thứ hai.

[ ] Gửi thành công thì toàn bộ biểu mẫu và trạng thái đã chạm được đặt lại về ban đầu.

Vấn đề gặp phải trong Lab 3

Vấn đề gặp phải	Nguyên nhân có thể	Cách khắc phục
Thông báo lỗi hiện ngay khi vừa mở trang, chưa động vào ô nào	Thiếu điều kiện daCham[ten] && khi hiển thị lỗi	Luôn gọi qua hàm loiHienThi() thay vì đọc thẳng loi.ten
Bấm nút Gửi khi biểu mẫu rỗng nhưng không thấy lỗi nào hiện ra	Quên đánh dấu tatCaDaCham trong xuLyGui	Gọi setDaCham(tatCaDaCham) trước khi kiểm tra điều kiện dừng

Vấn đề gặp phải	Nguyên nhân có thể	Cách khắc phục
Trang tải lại mỗi lần bấm Gửi, mất hết dữ liệu đang nhập	Thiếu `e.preventDefault()` đầu hàm xuLyGui	Thêm dòng đó làm câu lệnh đầu tiên trong hàm
Bấm Gửi hai lần liên tiếp trong lúc đang giả lập, tạo hai lần "thành công"	Nút gửi thiếu `disabled={trangThai === "dang-gui"}`	Thêm điều kiện khoá nút đúng như Bước 5

> **Từ đây trở đi: bài tập về nhà**
> Lab 4 và Lab 5 không làm trên lớp. Hoàn thành ở nhà, đẩy lên cùng nhánh feature/buoi8-form-co-kiem-soat, nộp theo đúng hạn ghi ở mục "Hướng dẫn nộp bài qua GitHub" cuối tài liệu này.

LAB 4 (VỀ NHÀ) — RÚT LOGIC VÀO HOOK useForm

⏱ **Thời gian ước tính: 40 phút | 🎯 Mục tiêu: đóng gói state, handler và kiểm chứng của Lab 1–3 vào một hook tùy biến, rồi chứng minh hook dùng lại được bằng một biểu mẫu thứ hai.**

Bước 1 — Tạo hook dùng chung

Tạo tệp `src/hooks/useForm.js`. Đây là bản tổng hợp lại toàn bộ logic đã viết tay ở Lab 1–3, không có gì mới về mặt kỹ thuật — chỉ là gói ghém cho gọn.

JS — src/hooks/useForm.js

```
import { useState } from 'react';

export function useForm(giaTriBanDau, kiemChung) {
  const [duLieu, setDuLieu] = useState(giaTriBanDau);
  const [daCham, setDaCham] = useState({});
  const [dangGui, setDangGui] = useState(false);

  const loi = kiemChung(duLieu);
  const hopLe = Object.keys(loi).length === 0;

  function xuLyThayDoi(e) {
    const { name, value, type, checked } = e.target;
    setDuLieu((truoc) => ({ ...truoc, [name]: type === 'checkbox' ? checked : value }));
  }

  function xuLyRoiO(e) {
    const { name } = e.target;
    setDaCham((truoc) => ({ ...truoc, [name]: true }));
  }

  function loiCuaO(ten) {
    return daCham[ten] ? loi[ten] : undefined;
  }

  function datLai() {
    setDuLieu(giaTriBanDau);
    setDaCham({});
    setDangGui(false);
  }

  function xuLyGui(guiDuLieu) {
    return async (e) => {
      e.preventDefault();
      const tatCaDaCham = {};
      Object.keys(giaTriBanDau).forEach((k) => { tatCaDaCham[k] = true; });
      setDaCham(tatCaDaCham);

      if (Object.keys(kiemChung(duLieu)).length > 0) return;

      try {
        setDangGui(true);
        await guiDuLieu(duLieu);
```

```
    } finally {
      setDangGui(false);
    }
  };
 }

 return { duLieu, loi, daCham, dangGui, hopLe, xuLyThayDoi, xuLyRoiO, loiCuaO, xuLyGui, datLai, setDuLieu };
}
```

Bước 2 — Chuyển FormThemDiaDiem sang dùng hook

Xoá state và các hàm đã viết tay ở Lab 1–3 trong FormThemDiaDiem.jsx, thay bằng một dòng gọi hook. Phần JSX giữ nguyên, chỉ đổi tên biến cho khớp.

JSX — src/features/dia-diem/FormThemDiaDiem.jsx (bản dùng hook)

```
import { useForm } from '../../hooks/useForm';
import { kiemChung } from './kiemChung';

const GIA_TRI_BAN_DAU = { ten: '', moTa: '', giaVe: '', phuong: '', loaiHinh: 'di-tich', dongY: false };

export default function FormThemDiaDiem({ onThemXong }) {
 const { duLieu, dangGui, xuLyThayDoi, xuLyRoiO, loiCuaO, xuLyGui, datLai } =
  useForm(GIA_TRI_BAN_DAU, kiemChung);

 const gui = xuLyGui(async (gt) => {
  await new Promise((r) => setTimeout(r, 1200));
  onThemXong({ ...gt, giaVe: Number(gt.giaVe) });
  datLai();
 });

 return (
  <form onSubmit={gui} noValidate>
   <input
    name="ten" value={duLieu.ten}
    onChange={xuLyThayDoi} onBlur={xuLyRoiO}
   />
   {loiCuaO('ten') && <p role="alert">{loiCuaO('ten')}</p>}
   {/* ... các ô còn lại theo đúng khuôn Lab 1–3 ... */}
   <button type="submit" disabled={dangGui}>
    {dangGui ? 'Đang lưu...' : 'Thêm địa điểm'}
   </button>
  </form>
 );
}
```

Bước 3 — Chứng minh hook dùng lại được: biểu mẫu Góp ý

Tạo tệp `src/features/gop-y/FormGopY.jsx` — một biểu mẫu hoàn toàn khác nội dung (góp ý về địa điểm) nhưng dùng lại chính hook `useForm`. Nếu Bước 1 làm đúng, tệp này sẽ rất ngắn.

JSX — src/features/gop-y/FormGopY.jsx

```
import { useForm } from '../../hooks/useForm';

const GIA_TRI_BAN_DAU = { hoTen: '', noiDung: '' };

function kiemChungGopY(d) {
 const loi = {};
 if (!d.hoTen.trim()) loi.hoTen = 'Vui lòng nhập họ tên.';
 if (d.noiDung.trim().length < 10) loi.noiDung = 'Góp ý cần ít nhất 10 ký tự.';
 return loi;
```

```
}

export default function FormGopY() {
 const { duLieu, dangGui, xuLyThayDoi, xuLyRoiO, loiCuaO, xuLyGui, datLai } =
  useForm(GIA_TRI_BAN_DAU, kiemChungGopY);

 const gui = xuLyGui(async (gt) => {
  await new Promise((r) => setTimeout(r, 800));
  alert('Cảm ơn góp ý của ' + gt.hoTen);
  datLai();
 });

 return (
  <form onSubmit={gui} noValidate>
   <input name="hoTen" value={duLieu.hoTen} onChange={xuLyThayDoi} onBlur={xuLyRoiO} />
   {loiCuaO('hoTen') && <p role="alert">{loiCuaO('hoTen')}</p>}

   <textarea name="noiDung" value={duLieu.noiDung} onChange={xuLyThayDoi} onBlur={xuLyRoiO} />
   {loiCuaO('noiDung') && <p role="alert">{loiCuaO('noiDung')}</p>}

   <button type="submit" disabled={dangGui}>Gửi góp ý</button>
  </form>
 );
}
```

Kết quả mong đợi

FormThemDiaDiem và FormGopY dùng chung một hook, không chép lại bất kỳ state hay handler nào.
Sửa một lỗi trong useForm.js (ví dụ cách đánh dấu daCham) -> cả hai biểu mẫu cùng được sửa.

✓ CHECKPOINT: Lab 4 hoàn thành

[ ] FormThemDiaDiem hoạt động y hệt như cuối Lab 3, nhưng không còn state hay handler viết tay trong chính component.

[ ] FormGopY chạy được độc lập, dùng đúng hook useForm, với luật kiểm chứng riêng của nó.

[ ] Không có đoạn logic nào (state, xuLyThayDoi, xuLyGui...) bị chép giữa hai component.

LAB 5 (VỀ NHÀ) — NÂNG TRẠNG THÁI LÊN CHO KHUNG XEM TRƯỚC

⏱ **Thời gian ước tính: 30 phút | 🎯 Mục tiêu: hiển thị một thẻ xem trước cập nhật theo từng ký tự đang gõ trong FormThemDiaDiem, bằng kỹ thuật nâng trạng thái lên thành phần cha chung.**

Bước 1 — Tạo thành phần cha giữ state

JSX — src/pages/TrangThemDiaDiem.jsx

```
import { useForm } from '../hooks/useForm';
import { kiemChung } from '../features/dia-diem/kiemChung';
import FormThemDiaDiem from '../features/dia-diem/FormThemDiaDiem';
import XemTruocTheDiaDiem from '../features/dia-diem/XemTruocTheDiaDiem';

const GIA_TRI_BAN_DAU = { ten: '', moTa: '', giaVe: '', phuong: '', loaiHinh: 'di-tich', dongY: false };

export default function TrangThemDiaDiem() {
  const form = useForm(GIA_TRI_BAN_DAU, kiemChung);

  return (
    <div className="bo-cuc-hai-cot">
      <FormThemDiaDiem {...form} />
      <XemTruocTheDiaDiem duLieu={form.duLieu} />
    </div>
  );
}
```

Muốn làm được việc này, `FormThemDiaDiem` phải đổi từ "tự gọi useForm bên trong" sang "nhận toàn bộ props từ cha". Đây chính là lý do state phải nâng lên: hai thành phần con (form và khung xem trước) đều cần đọc `duLieu`, nên không component con nào được giữ nó riêng cho mình nữa.

Bước 2 — Sửa FormThemDiaDiem để nhận props thay vì tự gọi hook

JSX — src/features/dia-diem/FormThemDiaDiem.jsx (bản nhận props)

```
export default function FormThemDiaDiem({ duLieu, xuLyThayDoi, xuLyRoiO, loiCuaO, xuLyGui, dangGui }) {
  const gui = xuLyGui(async (gt) => {
    await new Promise((r) => setTimeout(r, 1200));
    alert('Đã thêm: ' + gt.ten);
  });

  return (
    <form onSubmit={gui} noValidate>
      <input name="ten" value={duLieu.ten} onChange={xuLyThayDoi} onBlur={xuLyRoiO} />
      {loiCuaO('ten') && <p role="alert">{loiCuaO('ten')}</p>}
      {/* ... các ô còn lại ... */}
      <button type="submit" disabled={dangGui}>Thêm địa điểm</button>
    </form>
  );
}
```

Bước 3 — Thành phần xem trước, chỉ đọc

JSX — src/features/dia-diem/XemTruocTheDiaDiem.jsx

```
export default function XemTruocTheDiaDiem({ duLieu }) {
  return (
    <aside className="the-xem-truoc">
      <h3>{duLieu.ten || 'Tên địa điểm sẽ hiện ở đây'}</h3>
      <p>{duLieu.moTa || 'Mô tả ngắn sẽ hiện ở đây...'}</p>
```

```
      <p>
        Giá vé: {duLieu.giaVe ? Number(duLieu.giaVe).toLocaleString('vi-VN') + ' đ' : 'Chưa nhập'}
      </p>
      <p>Phường: {duLieu.phuong || 'Chưa chọn'}</p>
    </aside>
  );
}
```

Kết quả mong đợi

Gõ "Chùa Thiên Mụ" vào ô Tên
-> thẻ xem trước bên cạnh đổi tiêu đề thành "Chùa Thiên Mụ" NGAY LẬP TỨC, cùng nhịp với ô nhập

Gõ "50000" vào ô Giá vé
-> thẻ xem trước hiện "Giá vé: 50.000 đ"

Vì sao XemTruocTheDiaDiem không có state riêng

Nếu thành phần này tự khai báo useState để giữ một bản sao dữ liệu, nó sẽ lập tức lệch với biểu mẫu ngay khi người dùng gõ — quay lại đúng vấn đề "hai nguồn sự thật" đã học ở mục V của tài liệu lý thuyết. Vì vậy thành phần xem trước chỉ nhận duLieu qua props và không giữ gì cho riêng mình.

✓ CHECKPOINT: Lab 5 hoàn thành

[ ] Thẻ xem trước cập nhật đồng thời với ô nhập, không có độ trễ hay cần bấm nút nào.

[ ] FormThemDiaDiem không còn tự gọi useForm — toàn bộ nhận qua props từ TrangThemDiaDiem.

[ ] XemTruocTheDiaDiem không khai báo bất kỳ useState nào.

BẢNG TEST CASE TỰ KIỂM TRA

Test case	Thao tác kiểm tra	Kết quả mong đợi	Kết quả thực tế
TC01	Gõ vào ô Tên	Chữ hiện ngay theo từng ký tự	
TC02	Chọn một phường trong danh sách	state.phuong đổi đúng mã của phường đã chọn	
TC03	Bấm nút chọn "Điểm ẩm thực" sau khi đã chọn "Di tích lịch sử"	Chỉ còn "Điểm ẩm thực" được chọn	
TC04	Tích hai tiện ích rồi bỏ tích một	Mảng tiện ích chỉ còn đúng tiện ích chưa bỏ	
TC05	Rời ô Tên khi đang để trống	Hiện thông báo "Vui lòng nhập tên địa điểm."	
TC06	Bấm Gửi khi biểu mẫu còn trống hoàn toàn	Toàn bộ lỗi hiện ra, không có yêu cầu máy chủ nào được gọi	
TC07	Điền đủ và hợp lệ, bấm Gửi	Nút đổi "Đang lưu...", khoá trong 1.2 giây, sau đó báo thành công	
TC08	Bấm Gửi hai lần liên tiếp thật nhanh	Chỉ một lượt gửi được thực hiện	
TC09 ⭐	Dùng FormGopY (Lab 4) với dữ liệu hợp lệ	Gửi thành công, không lỗi Console	
TC10 ⭐	Gõ vào ô Tên trong TrangThemDiaDiem (Lab 5)	Thẻ xem trước cập nhật cùng nhịp, không trễ	

BẢNG TỰ ĐÁNH GIÁ BÀI 8

Kỹ năng	Chưa / Cơ bản / Thành thạo	Ghi chú
Giải thích được vì sao cần ô có kiểm soát	☐ Chưa ☐ Cơ bản ☐ Thành thạo	
Viết một handler xử lý nhiều ô nhập bằng khoá tính toán được	☐ Chưa ☐ Cơ bản ☐ Thành thạo	
Phân biệt đúng value và checked cho từng loại ô nhập	☐ Chưa ☐ Cơ bản ☐ Thành thạo	
Viết hàm kiểm chứng thuần và hiển thị lỗi theo trạng thái đã chạm	☐ Chưa ☐ Cơ bản ☐ Thành thạo	
Quản lý trạng thái gửi biểu mẫu bằng biến nhiều giá trị	☐ Chưa ☐ Cơ bản ☐ Thành thạo	
Rút logic biểu mẫu vào hook tùy biến dùng lại được	☐ Chưa ☐ Cơ bản ☐ Thành thạo	
Nâng trạng thái lên khi hai thành phần cần chung dữ liệu	☐ Chưa ☐ Cơ bản ☐ Thành thạo	

✓ TÓM TẮT NHỮNG GÌ ĐÃ HỌC TRONG BÀI 8

- Dựng được một biểu mẫu nhiều ô nhập, hoàn toàn có kiểm soát, dùng một state đối tượng và một handler duy nhất.
- Xử lý đúng từng loại ô nhập: văn bản, vùng văn bản, danh sách chọn, hộp kiểm đơn, nhóm nút chọn, nhóm hộp kiểm nhiều lựa chọn.
- Viết hàm kiểm chứng dữ liệu thuần, tách khỏi giao diện, và điều khiển thời điểm hiển thị lỗi bằng trạng thái đã chạm.
- Quản lý trạng thái gửi biểu mẫu một cách an toàn, không mất dữ liệu người dùng khi có lỗi.
- Đóng gói toàn bộ logic vào hook tùy biến useForm, dùng lại được cho nhiều biểu mẫu khác nhau trong dự án.
- Đồng bộ hai thành phần anh em bằng kỹ thuật nâng trạng thái lên, thay vì để mỗi thành phần tự giữ một bản dữ liệu riêng.

HƯỚNG DẪN NỘP BÀI QUA GITHUB

1. Kiểm tra lại: `npm run dev` chạy sạch, không còn cảnh báo màu đỏ nào trong Console.
2. Thêm toàn bộ thay đổi: `git add .`
3. Ghi chú commit rõ ràng theo từng lab, ví dụ: `git commit -m "Buoi 8: form co kiem soat + kiem chung + useForm hook"`
4. Đẩy nhánh lên: `git push origin feature/buoi8-form-co-kiem-soat`
5. Mở Pull Request vào `main`, tiêu đề "Bài 8 — Họ tên — Mã sinh viên", mô tả ngắn những gì đã làm được trong Lab 1–5.
6. Hạn nộp: trước 23h59 ngày trước Bài 9. Giảng viên sẽ clone về và chạy npm run dev để chấm — dự án không chạy được coi như chưa nộp.